

Lecture 10

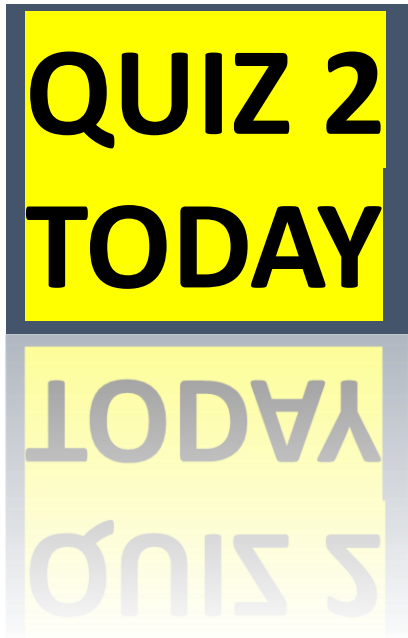
EE 421 / CS 425

Digital System Design

Fall 2026
Shahid Masud

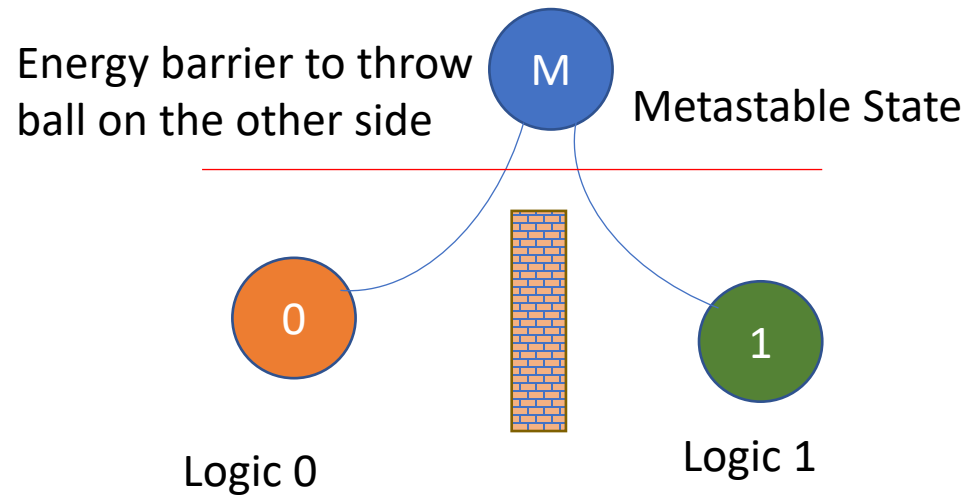
Topics

- Metastability
- Synchronizers for capturing Asynchronous Inputs
- Signal Transfer with and without Clock
- Communication between modules in Complex Digital System
- -----
- Computer Arithmetic
- Adder circuit
 - Ripple Carry Adder



Metastability, and Asynchronous Data Management

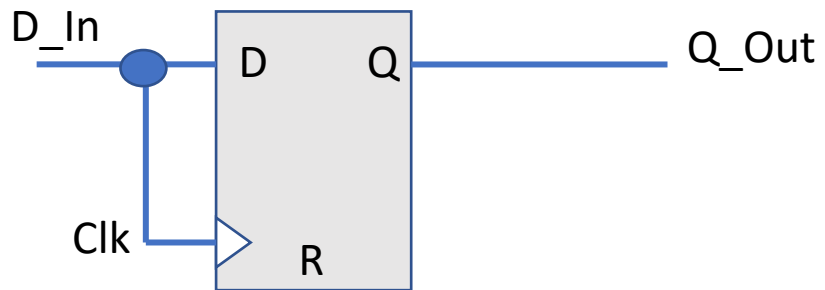
Objective – to reduce possible **Metastability** in capturing **Asynchronous Input**



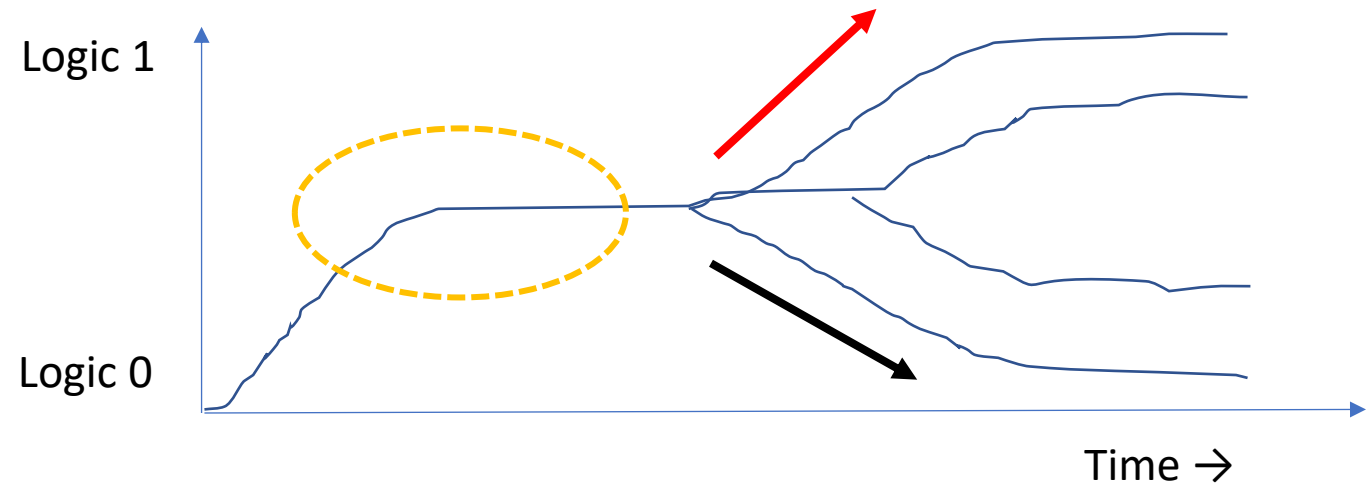
Synchronizer Failure: When flipflop hangs in a Metastable State for a long time (indefinitely)

Normally, the flipflop output would settle to a stable 0 or 1 state after some time

Output Behaviour with Metastability



**DFF is connected to produce metastability
As setup time is violated**



Oscilloscope trace of metastable behaviour

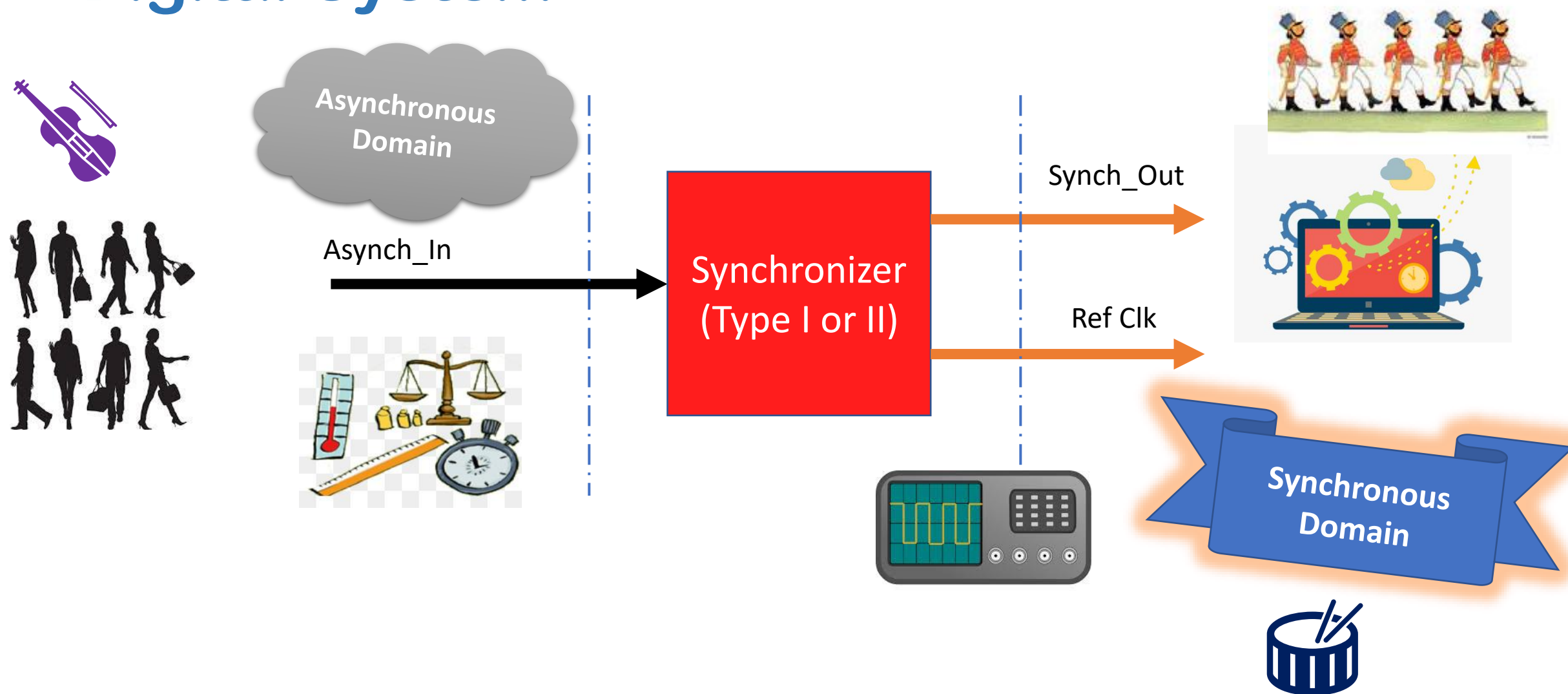
Eventually a stable state is reached

**Problem occurs when flipflop is not stable within
One clock period**

Synchronizers

- Asynchronous inputs are problematic as their transitions are not predictable
- High speed digital circuits rely on synchronizers to create a time buffer for recovering from a metastable event; thus reducing the possibility that metastability will cause circuit to malfunction
- An asynchronous signal should be synchronized by one synchronizer only. If not, then multiple synchronized signals could be present in the system and one of these could be driven into metastable state

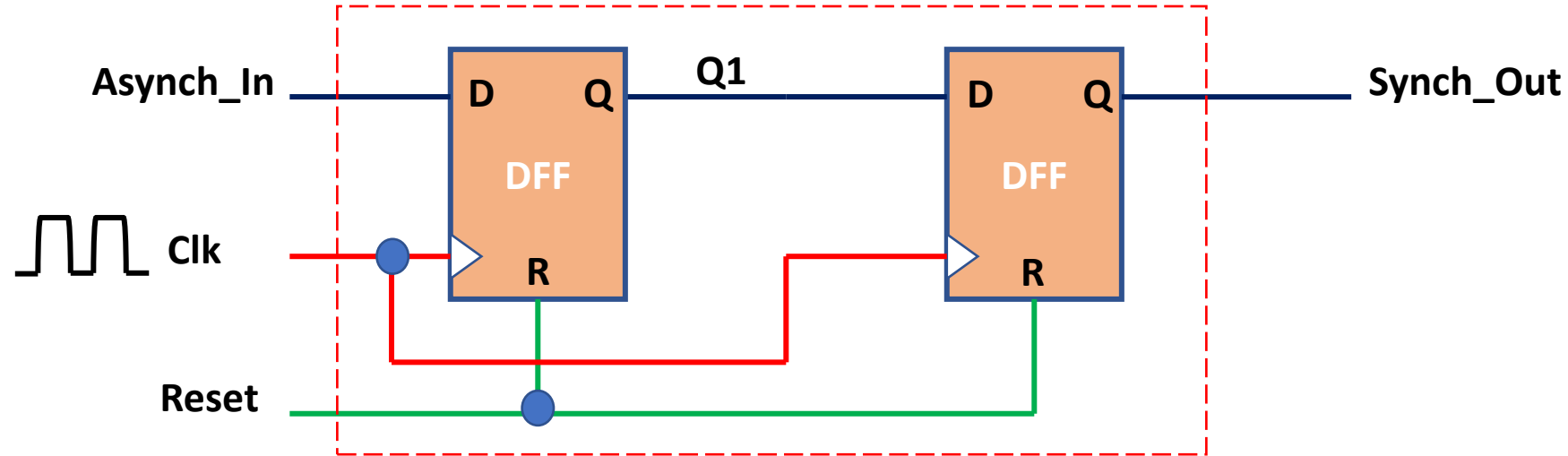
Asynchronous Inputs to a Synchronous Digital System



Synchronizer 1

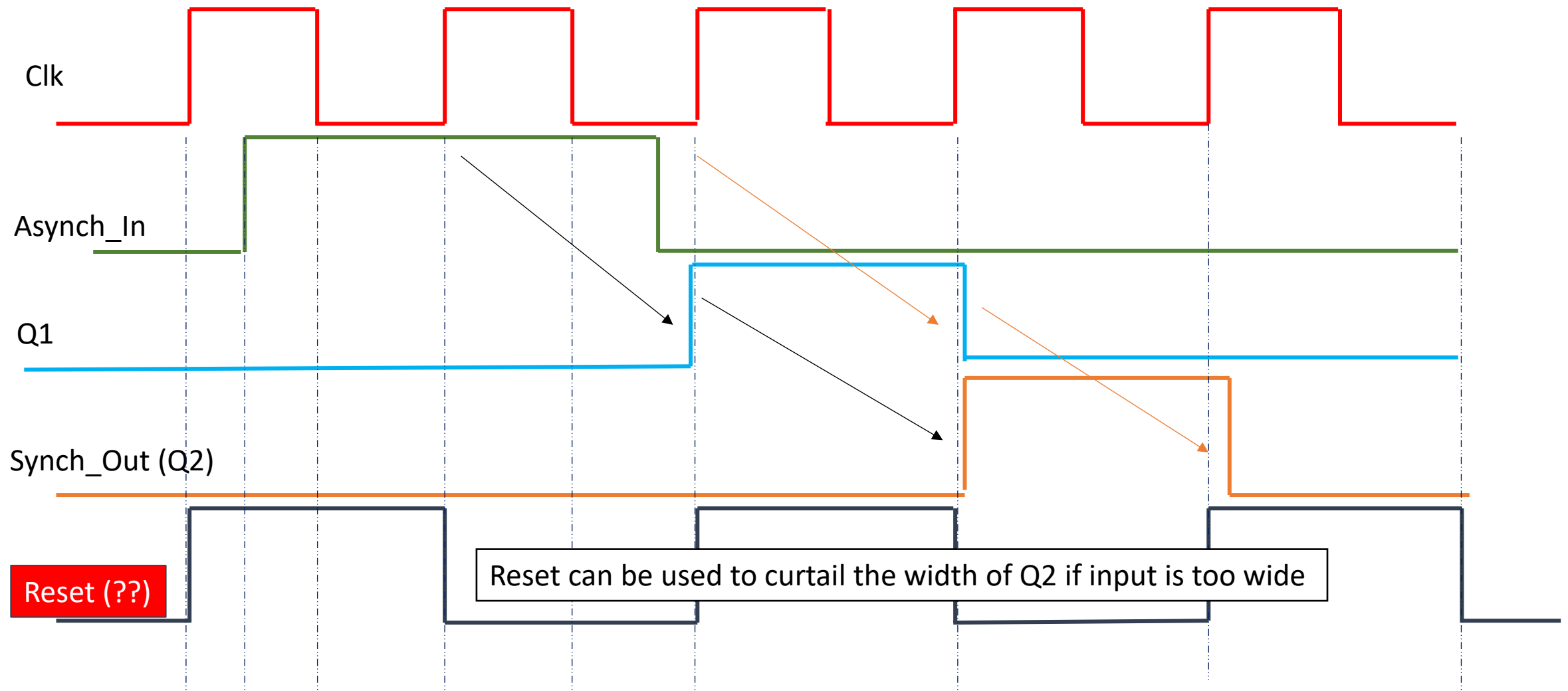
Condition: The width of asynchronous input pulse is greater than period of the clock

Synchronizer is a multistage shift register



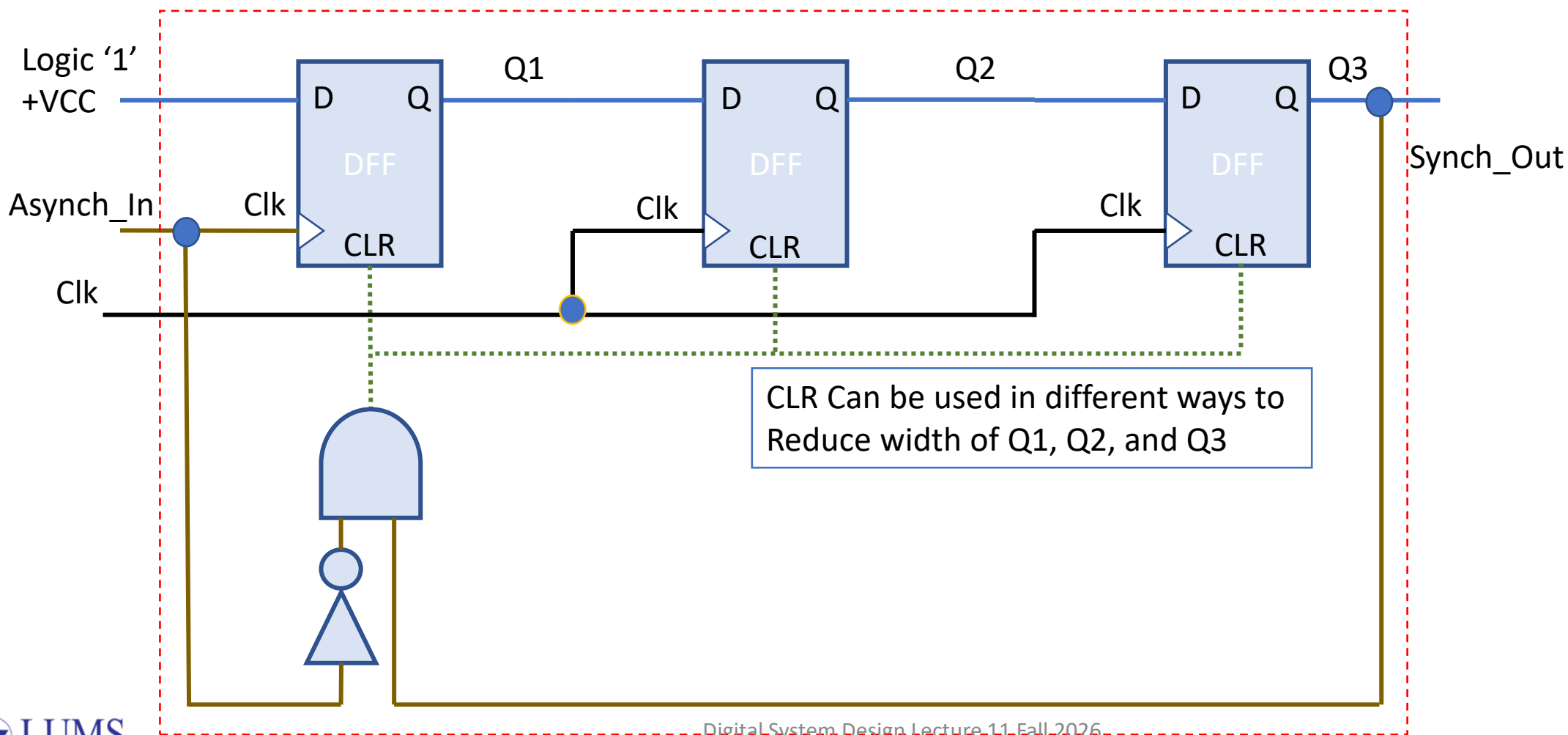
Reset Input 'R' is used as control to bring Synch_Out back to '0', as required

Timing Diagram – Synchronizer 1

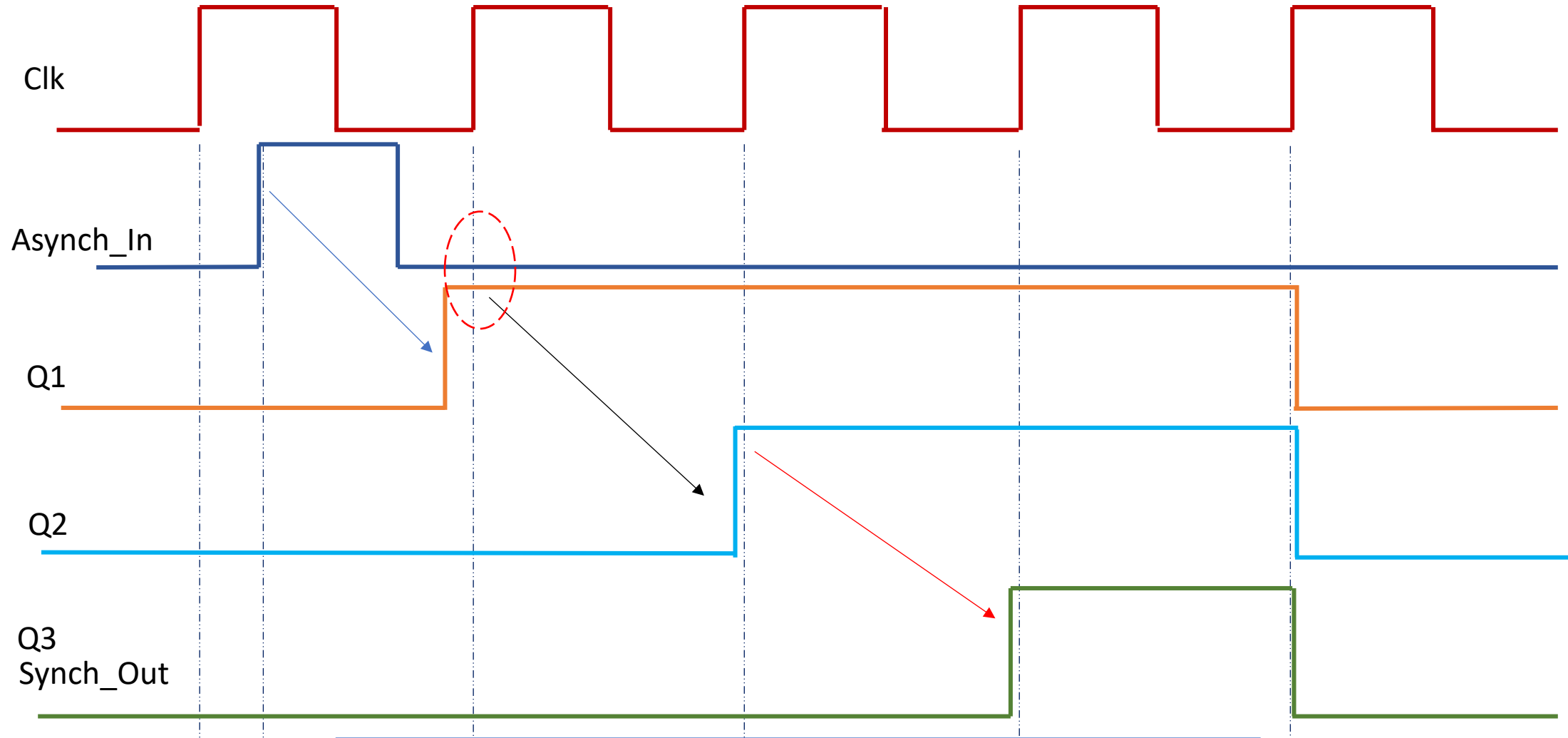


Synchronizer 2

Condition: Width of the asynchronous input pulse is less than the period of the clock



Timing Diagram – Synchronizer 2

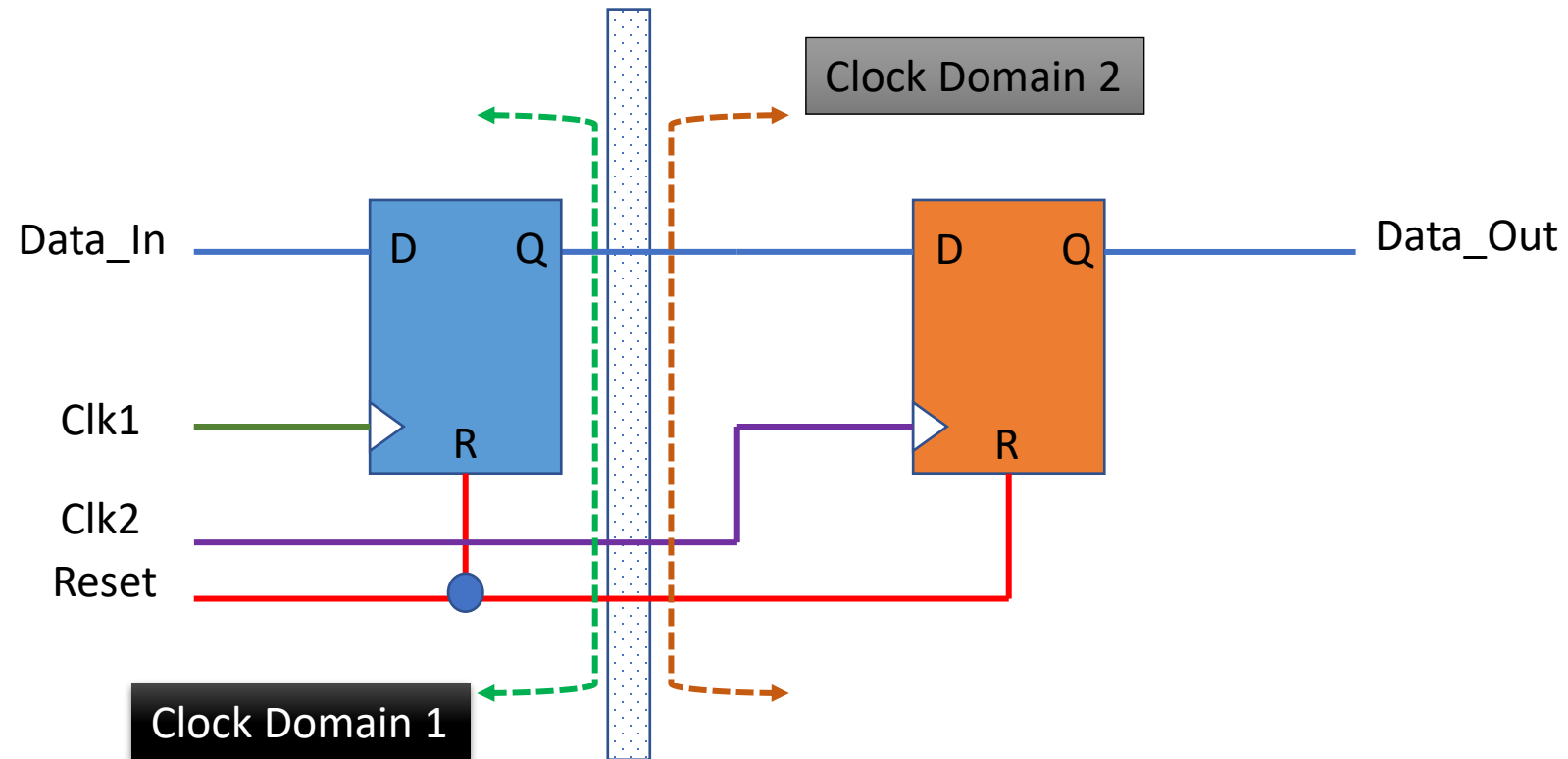


CLR input at DFF1 is used to bring back Q1 and then Q2 to Zero

Self-Timed and Speed Independent Circuits

- Having a completed Synchronous system is at times too challenging for a complex and fast digital system
- The limiting problem becomes how to distribute a single global clock without introducing intolerable clock skew
- The alternate is to partition the digital system into locally clocked pieces that communicate with each other using delay-insensitive signaling techniques (i.e. local clock for local communication)
- Each block proceeds at its own speed without the need for a global clock, synchronizing local communication whenever needed
- Usually a Request-Response Signalling method is employed

Data Reading across two Clock Domains



frequency of Clk1 is less than Clk2
 Otherwise, the **Synchronizer-2** is versatile and can be used here

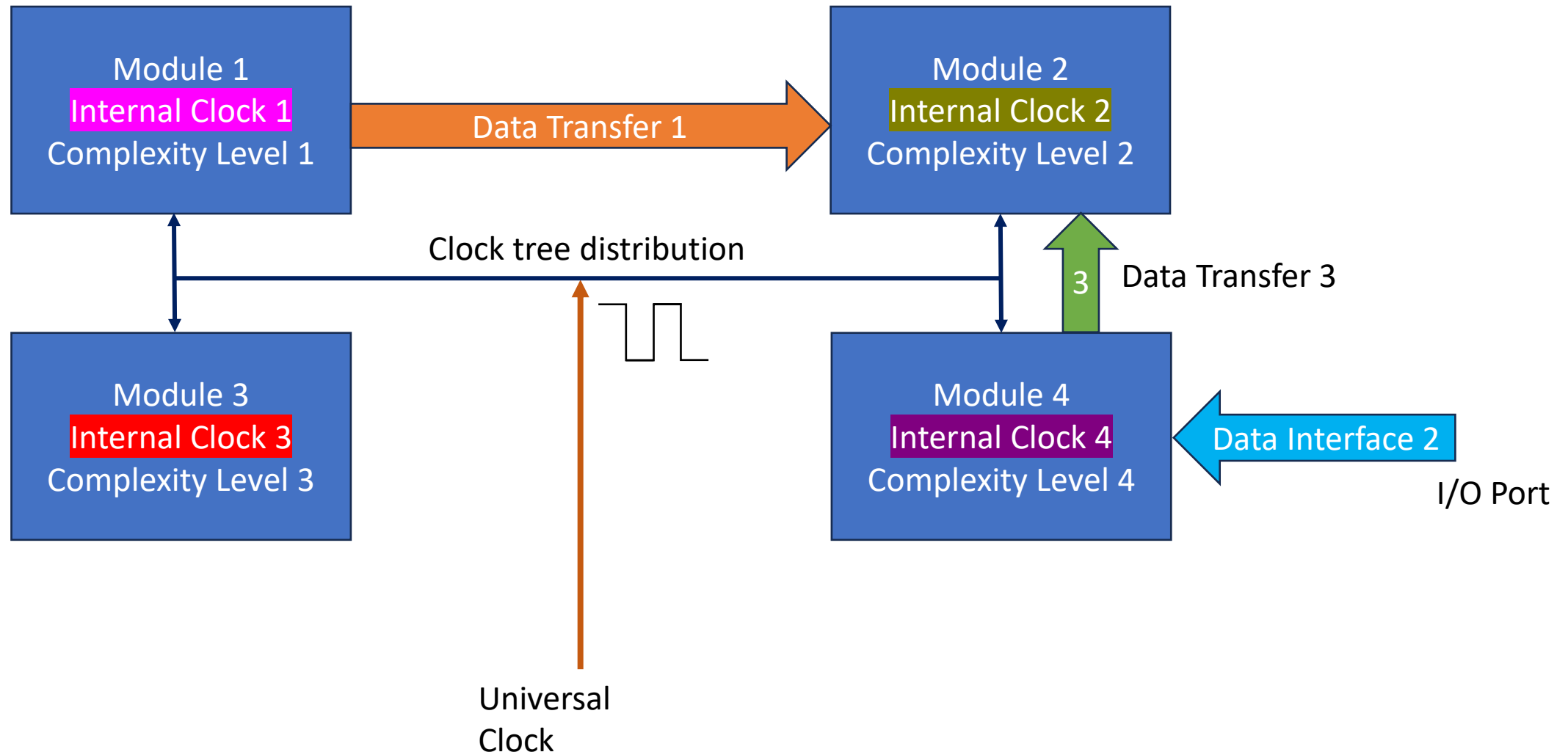
Communication across modules in Complex SoC

- Communication Signaling across modules at different clock speeds
- Self-timed circuits

Self-Timed and Speed Independent Circuits

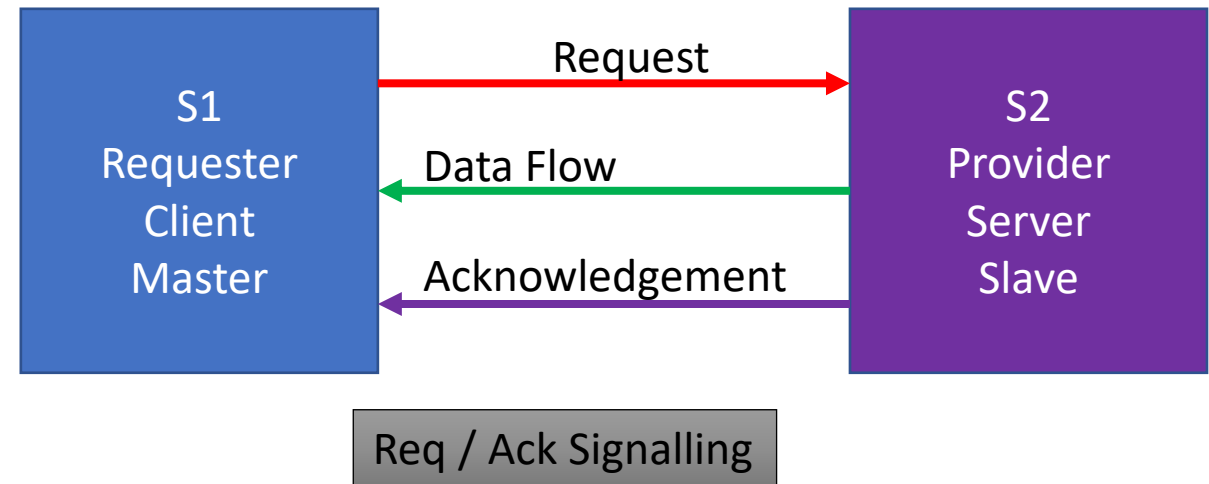
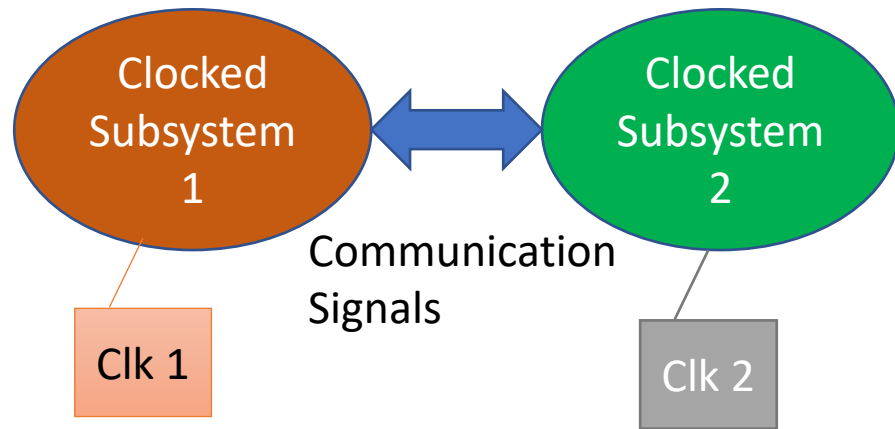
- Having a completed Synchronous system is at times too challenging for a complex and fast digital system
- The limiting problem becomes how to distribute a single global clock without introducing intolerable clock skew
- The alternate is to partition the digital system into locally clocked pieces that communicate with each other using delay-insensitive signaling techniques (i.e. local clock for local communication)
- Each block proceeds at its own speed without the need for a global clock, synchronizing local communication whenever needed
- Usually a Request-Response Signaling method is employed

Data Transfer in Complex Modular Design

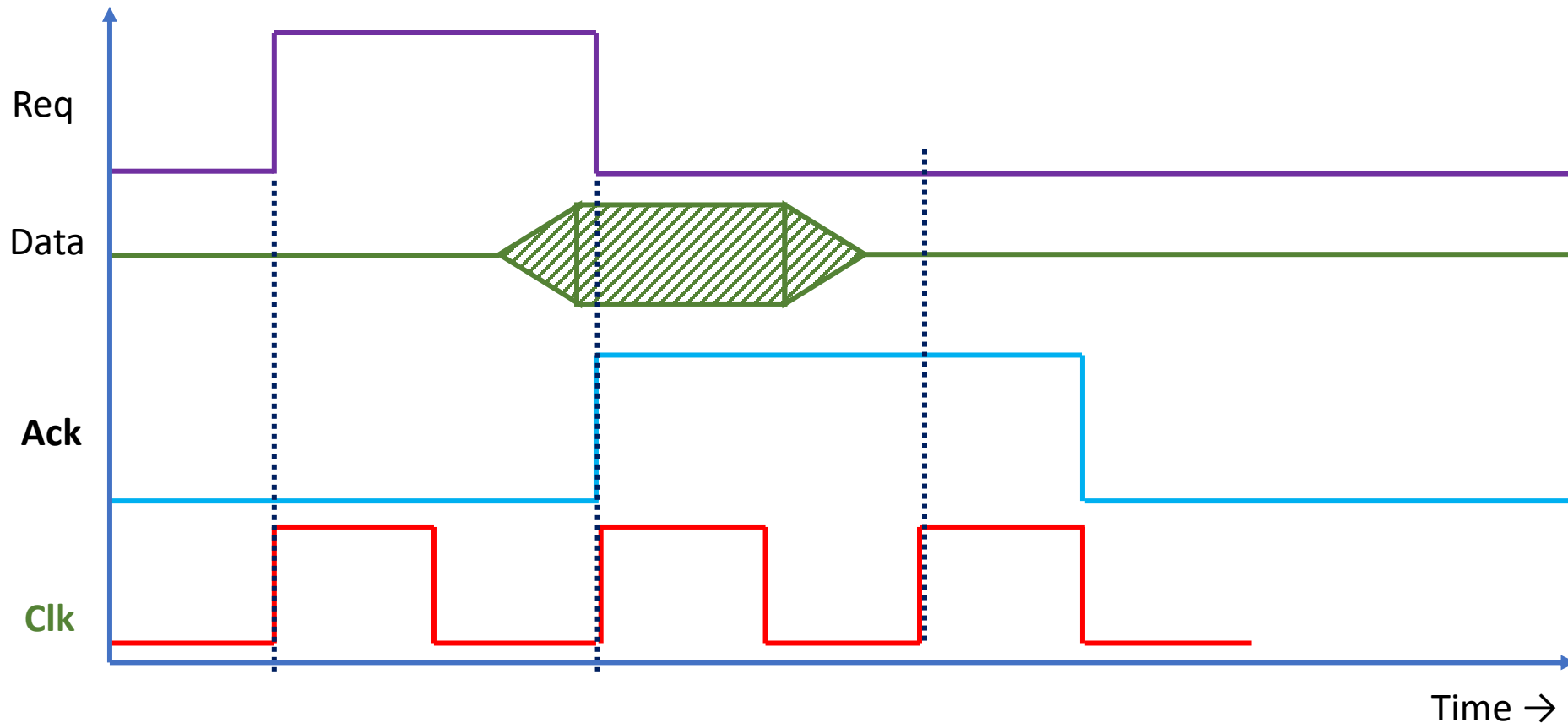


Request / Acknowledge Signaling

Independently clocked Subsystems

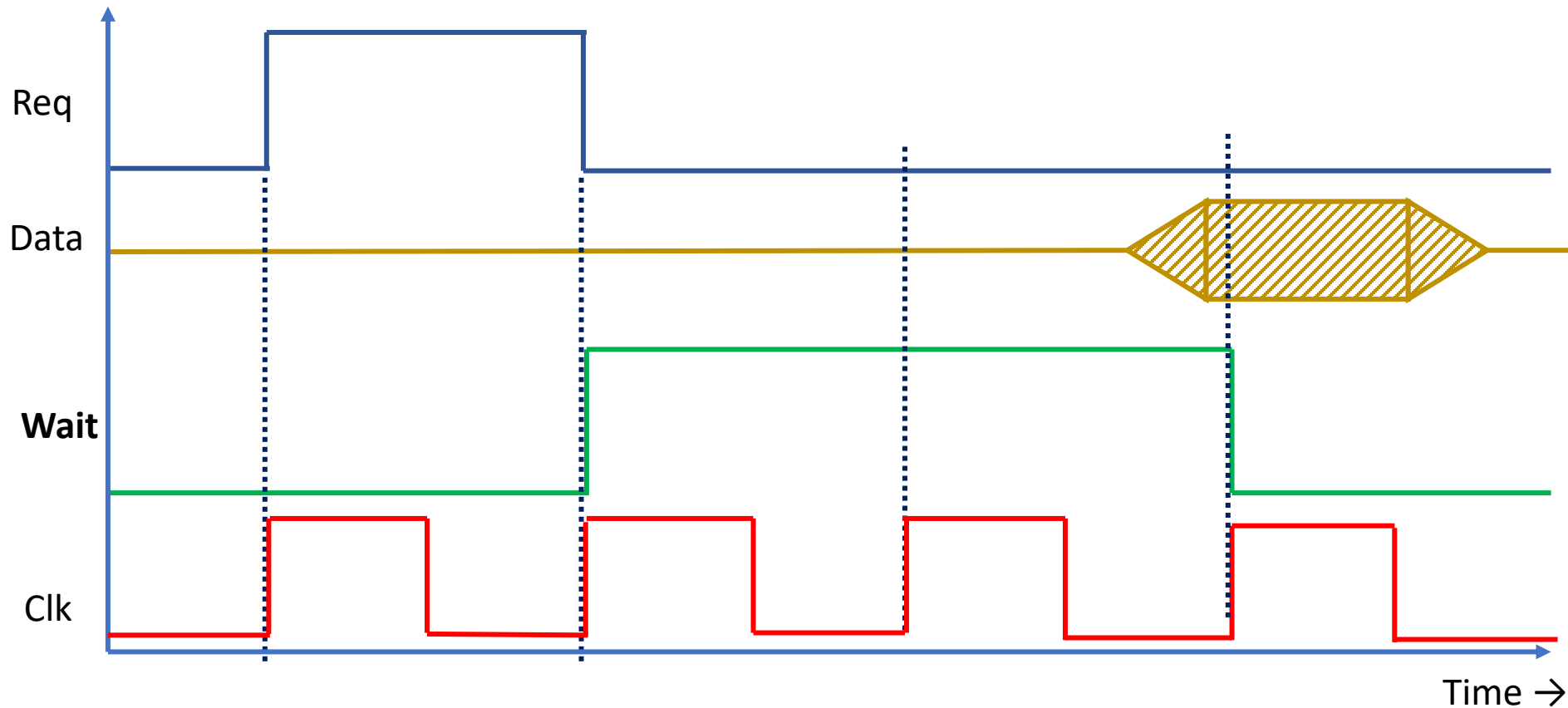


Synchronous Req / Ack Signaling



Req and Ack signals are synchronized to **Clk**

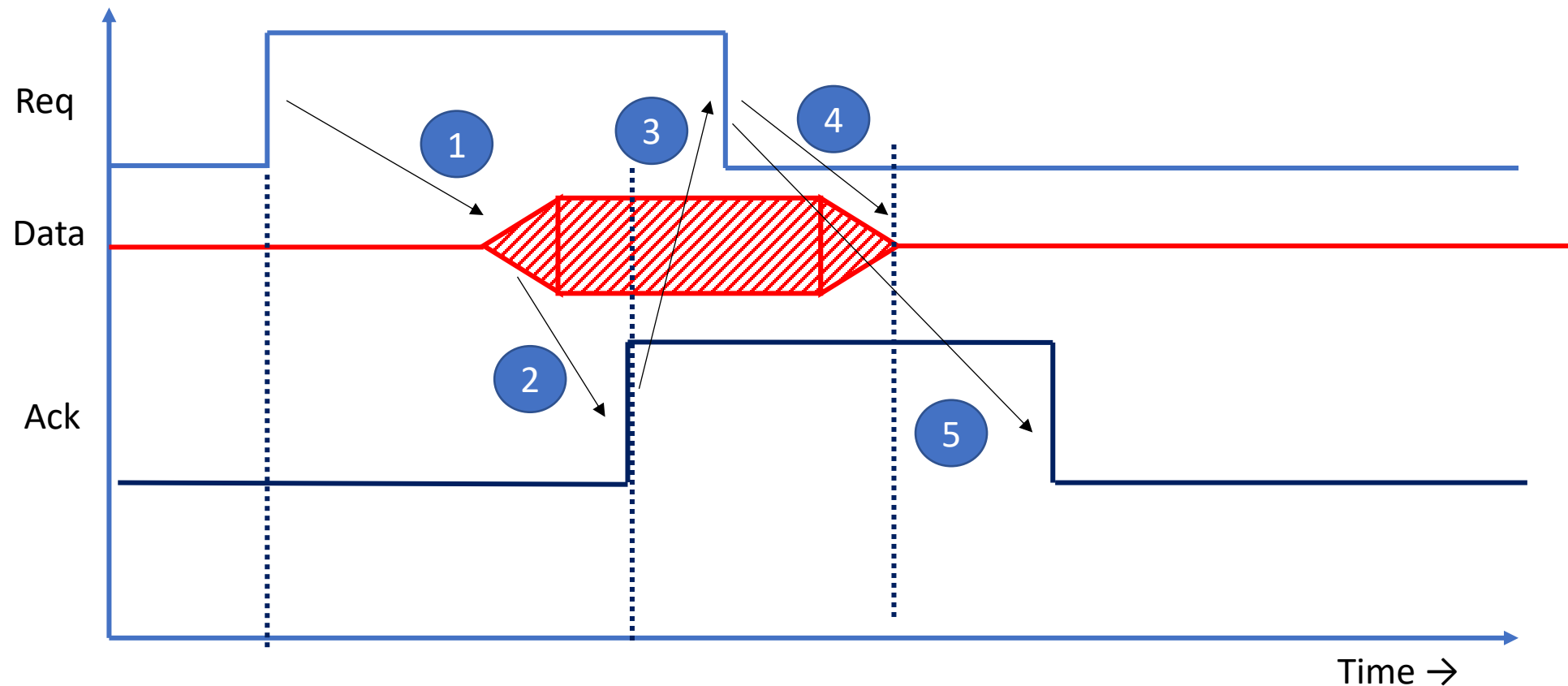
Synchronous Req with Wait Signaling



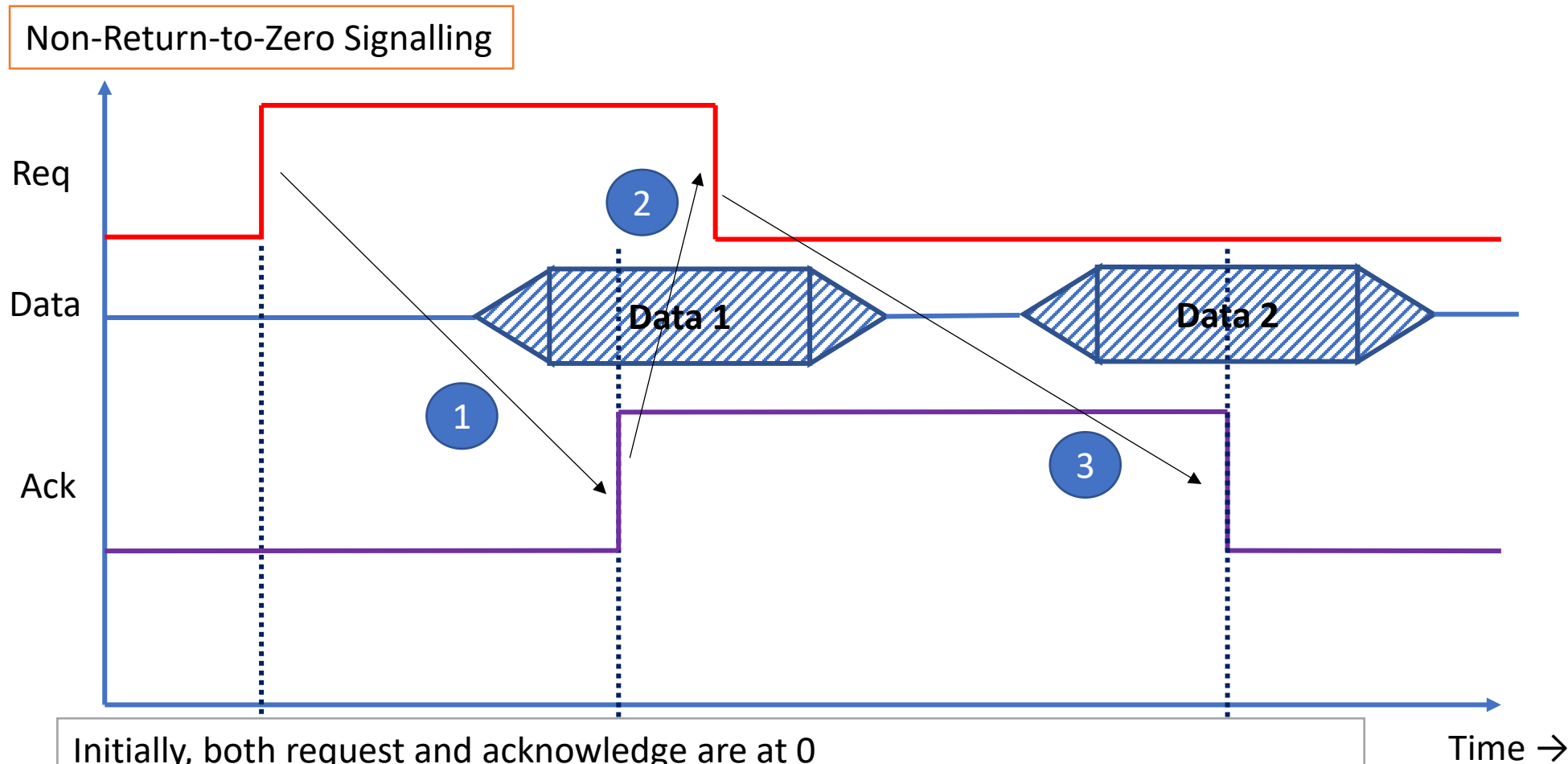
Slave can delay the Master by asserting Wait signal as it prepares the data and needs more clock cycles
 When the slave un-asserts Wait signal, it acknowledges that data is now available for the Master to read
 All interface signals are synchronized with Clock edge

Four Cycle Asynchronous Signaling

RTZ – Return to Zero Signaling with No Clock



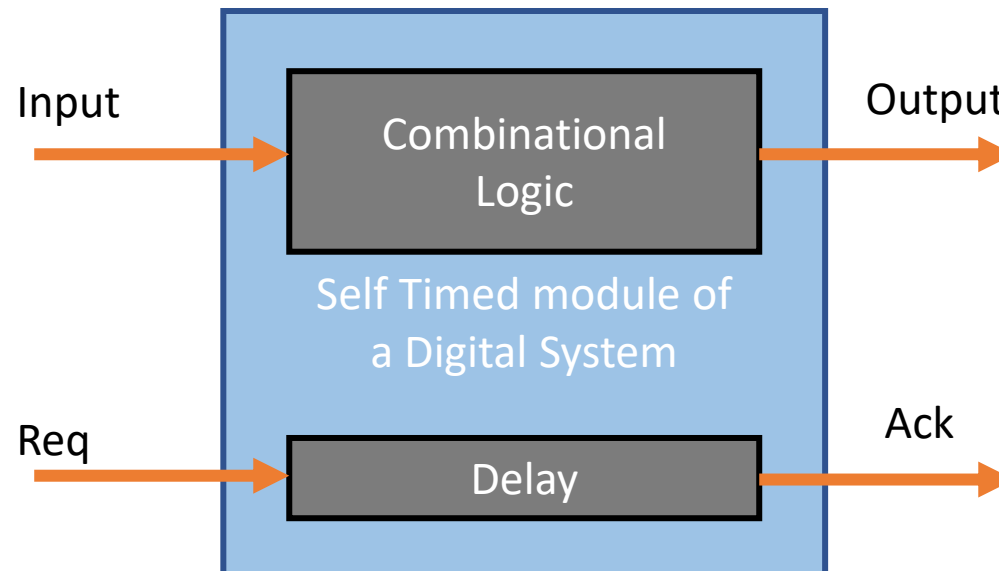
Two Cycle Asynchronous Signaling



Initially, both request and acknowledge are at 0
 Request line is complemented (to 1)
 Slave notices change in Req, provides data, and complements the Ack line (to 1)
 Further request and acknowledgement is by complementing the respective state

Self-Timed Circuits

- A self-timed circuit can determine on its own when a request has been serviced by mimicking the worst-case propagation delay path by using special logic to delay the request signal.
- This guarantees sufficient time to compute the correct output.



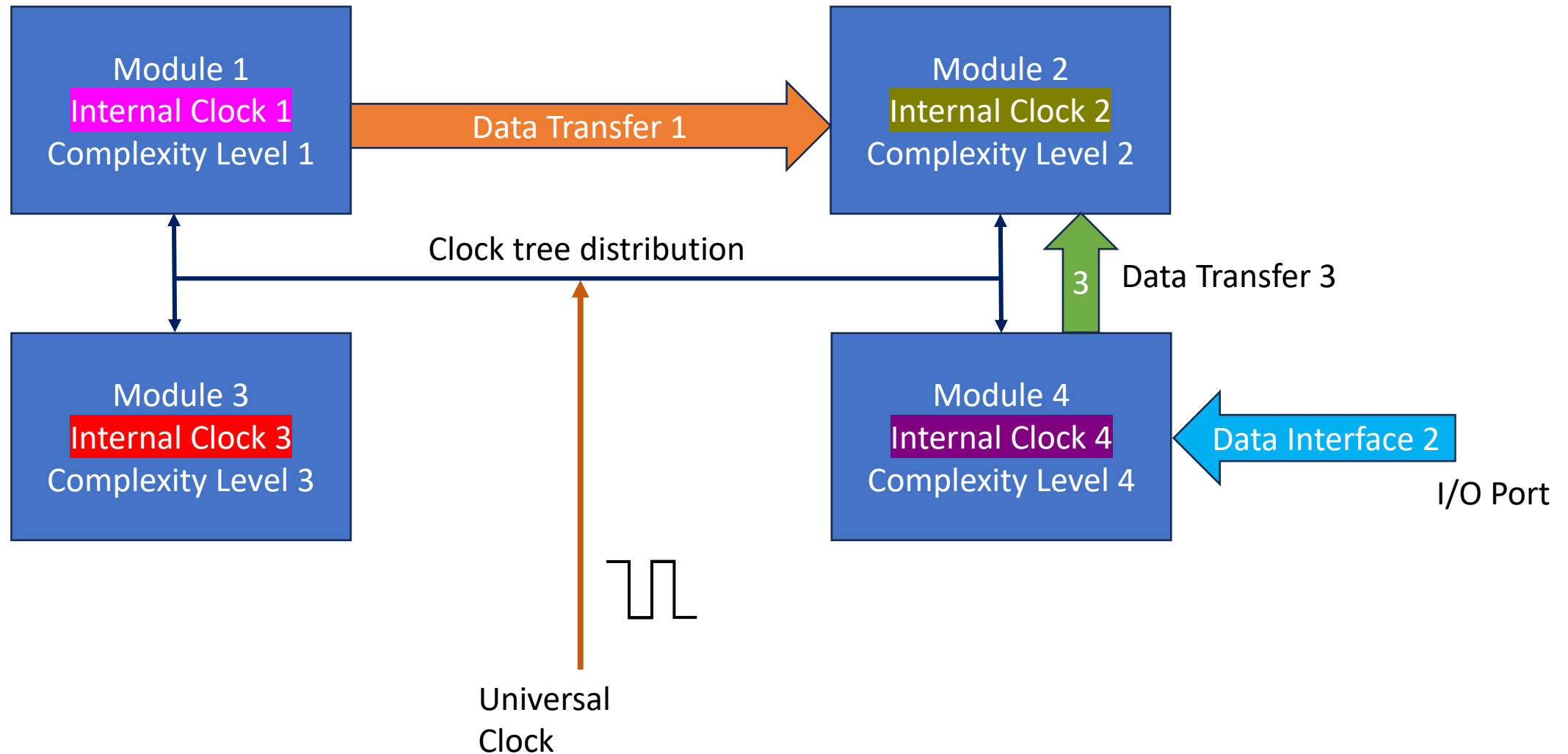
Topics

- Communication Signaling across modules at different clock speeds
(Revision – How clock is good and can be bad?) **Quick Review**
- -----
- Computer Arithmetic Circuits
- Basic Element – Full Adder
- Ripple Carry Adder – Timing Issues
- Bit Serial Adder – Concept of Throughput and Latency
- Carry Look Ahead Adder
- Carry Select Adder

Self-Timed and Speed Independent Circuits

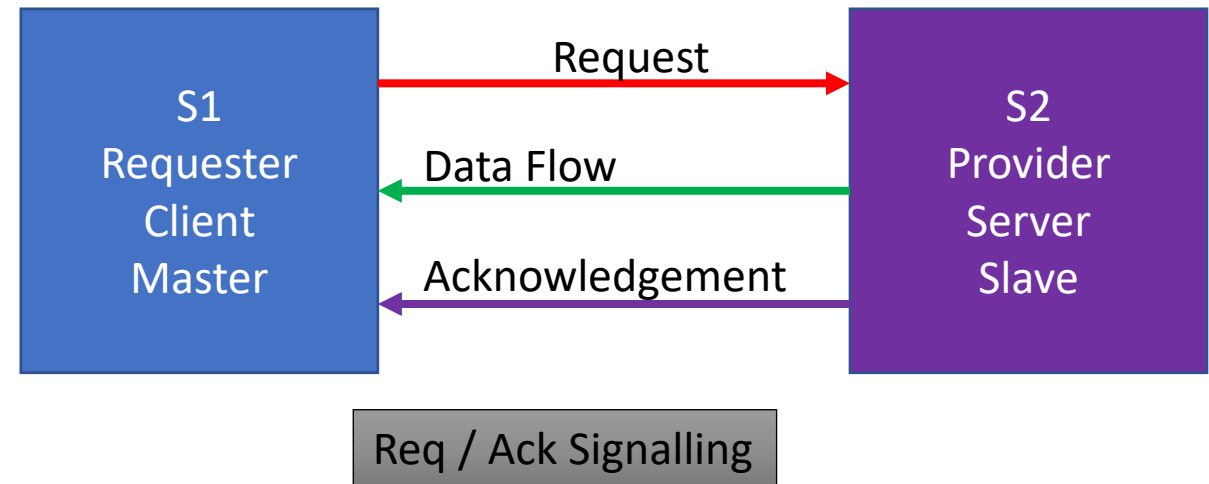
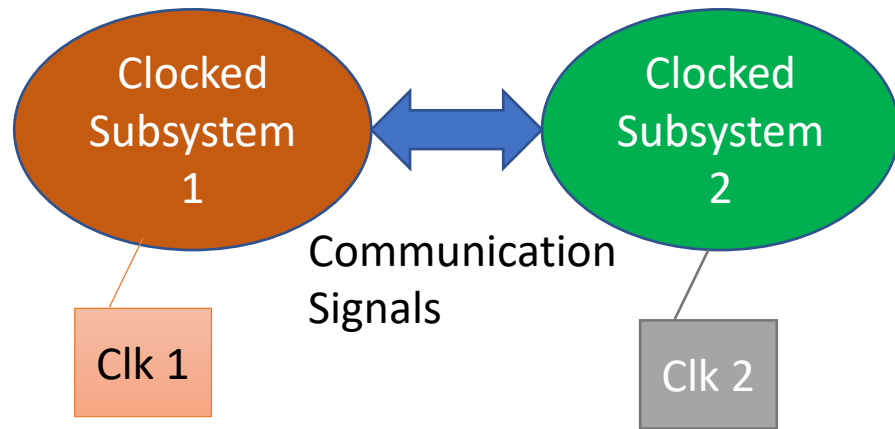
- Having a completed Synchronous system is at times too challenging for a complex and fast digital system
- The limiting problem becomes how to distribute a single global clock without introducing intolerable clock skew
- The alternate is to partition the digital system into locally clocked pieces that communicate with each other using delay-insensitive signaling techniques (i.e. local clock for local communication)
- Each block proceeds at its own speed without the need for a global clock, synchronizing local communication whenever needed
- Usually a Request-Response Signaling method is employed

Data Transfer in Complex Modular Design

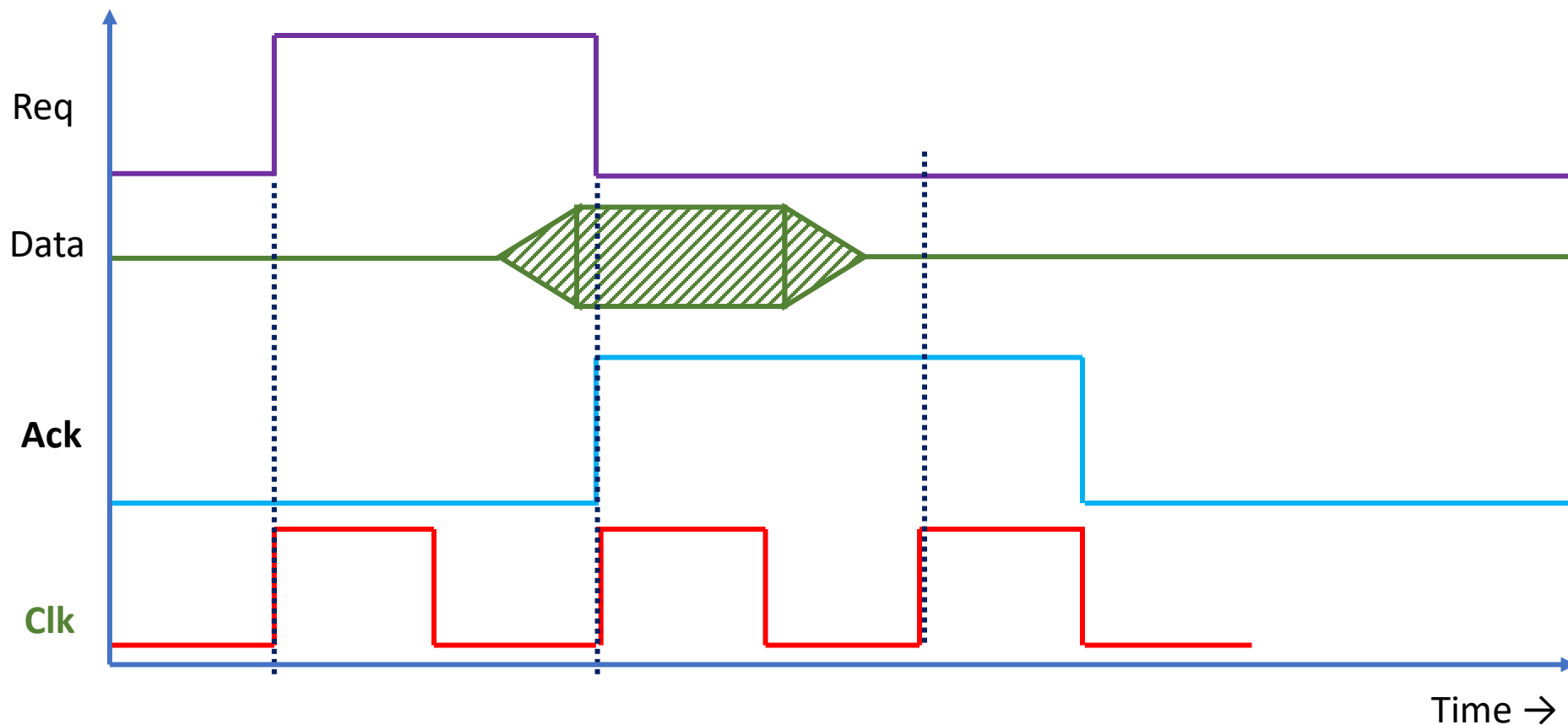


Request / Acknowledge Signaling

Independently clocked Subsystems

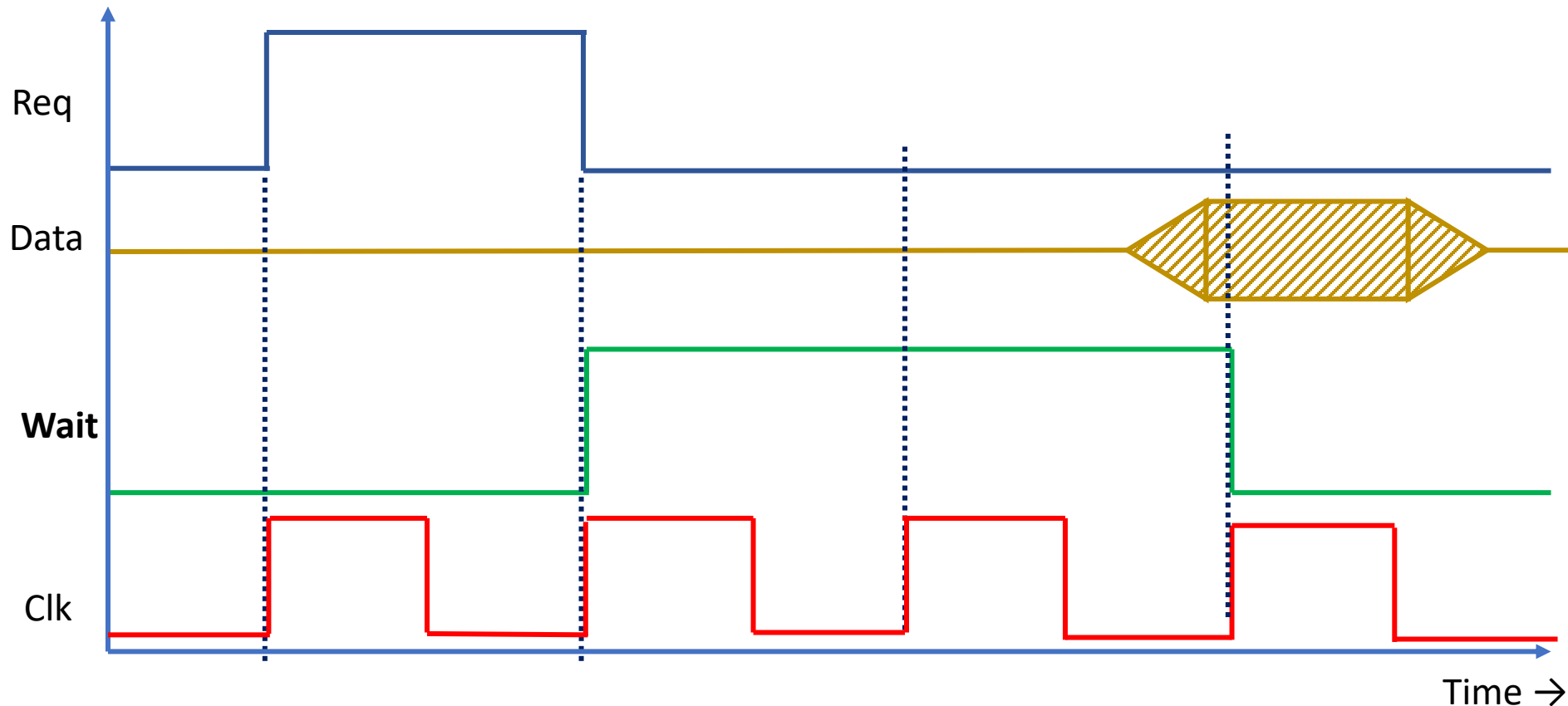


Synchronous Req / Ack Signaling



Req and Ack signals are synchronized to **Clk**

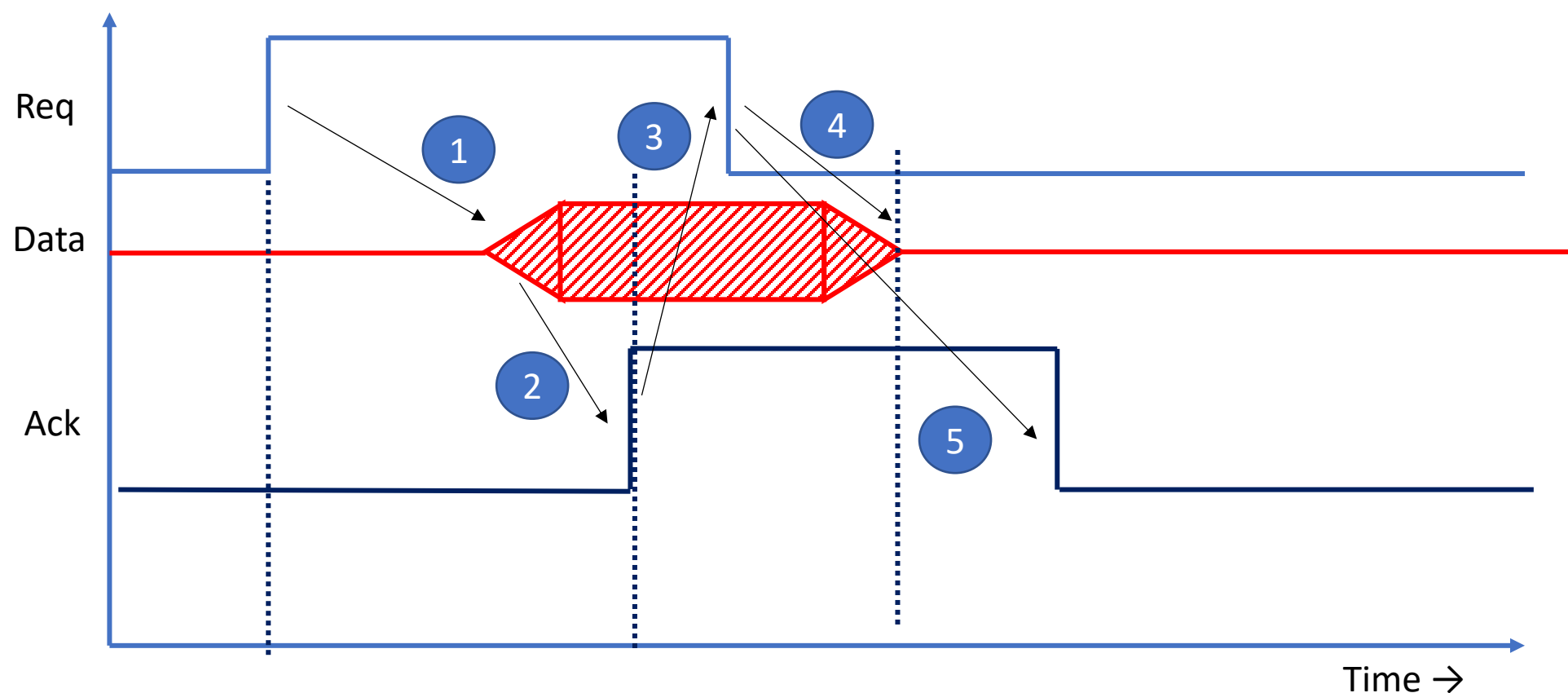
Synchronous Req with Wait Signaling



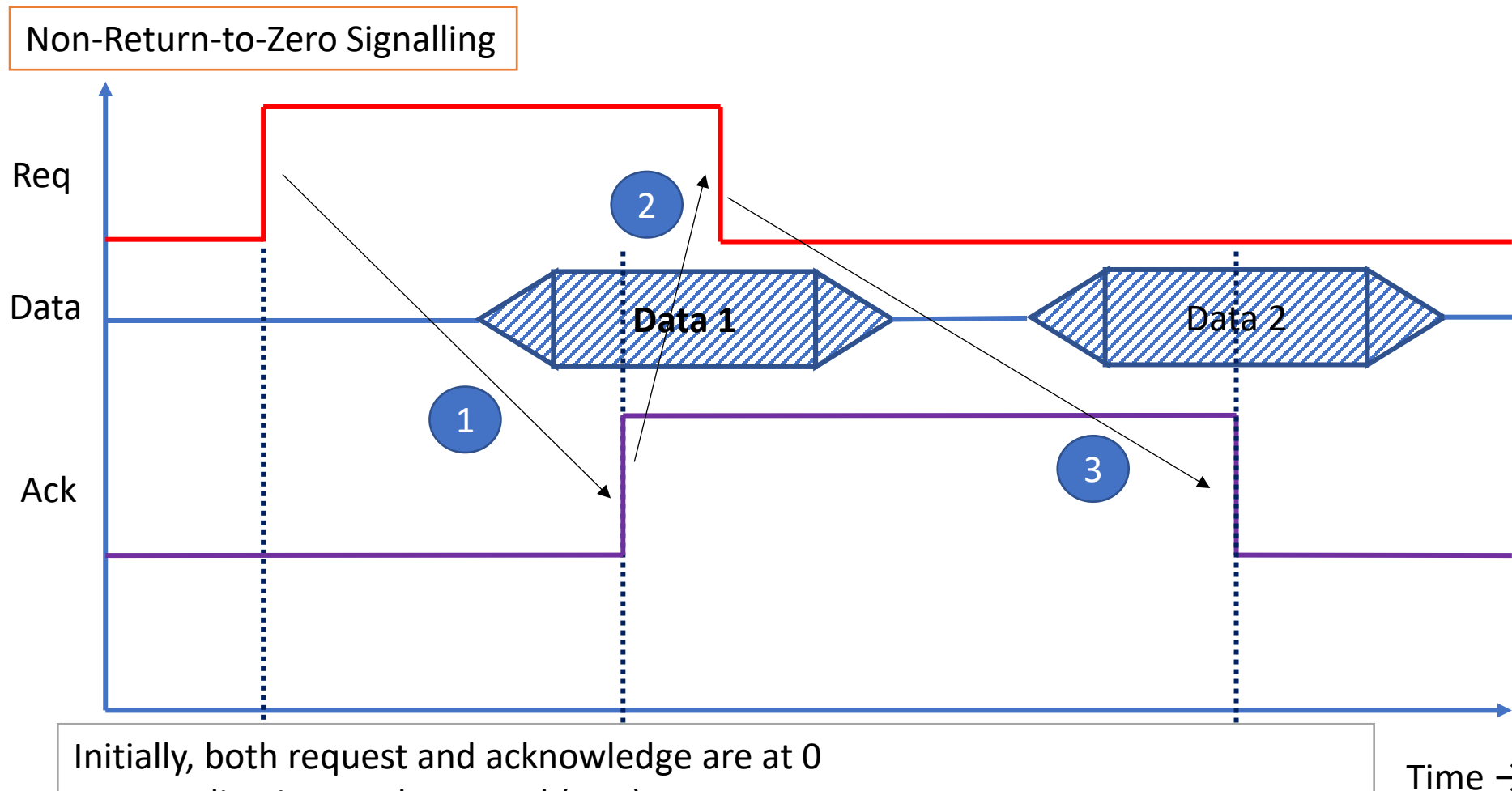
Slave can delay the Master by asserting Wait signal as it prepares the data and needs more clock cycles
 When the slave un-asserts Wait signal, it acknowledges that data is now available for the Master to read
 All interface signals are synchronized with Clock edge

Four Cycle Asynchronous Signaling

RTZ – Return to Zero Signaling with No Clock



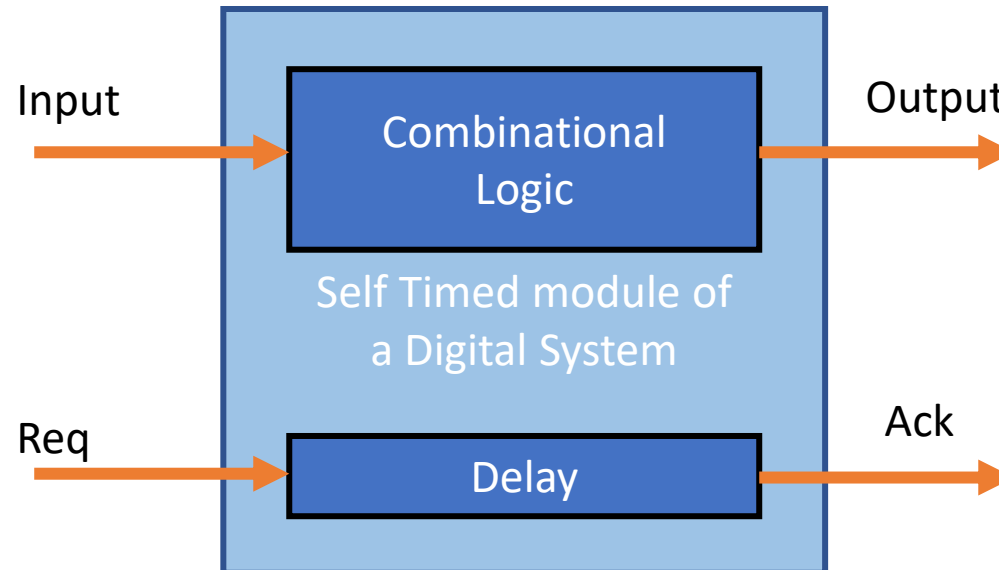
Two Cycle Asynchronous Signaling



Initially, both request and acknowledge are at 0
 Request line is complemented (to 1)
 Slave notices change in Req, provides data, and complements the Ack line (to 1)
 Further request and acknowledgement is by complementing the respective state

Self-Timed Circuits

- A self-timed circuit can determine on its own when a request has been serviced by mimicking the worst-case propagation delay path by using special logic to delay the request signal.
- This guarantees sufficient time to compute the correct output.



Computer Arithmetic

Computer Arithmetic

Study Data path and Control path of a binary hardware arithmetic unit

Computer Arithmetic Circuits

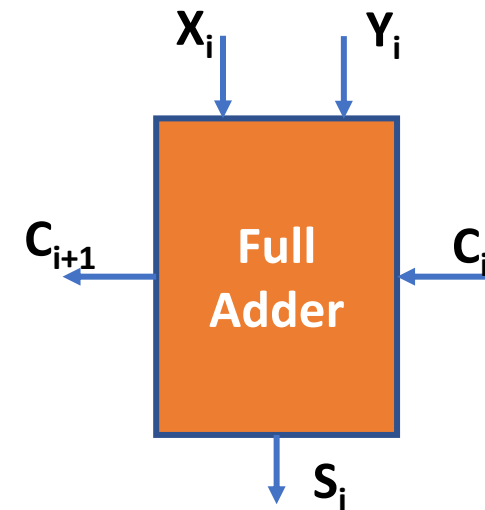
- Generally, the digital system design falls into one of the following categories:
 - **Data handling** – acquisition, storage, retrieval, compression, search, etc.
 - **Data processing** – filtering, de-noising, object detection, pattern recognition
- Computer Arithmetic is an important functional block of both aspects, especially plays an important part in Data Processing
- Efficient arithmetic circuits can improve the **speed – area – energy performance nexus** in a complex digital system

Basic Functional Block – Full Adder

X_i	Y_i	C_i	C_{i+1}	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\text{Sum } S_i = X_i \oplus Y_i \oplus C_i$$

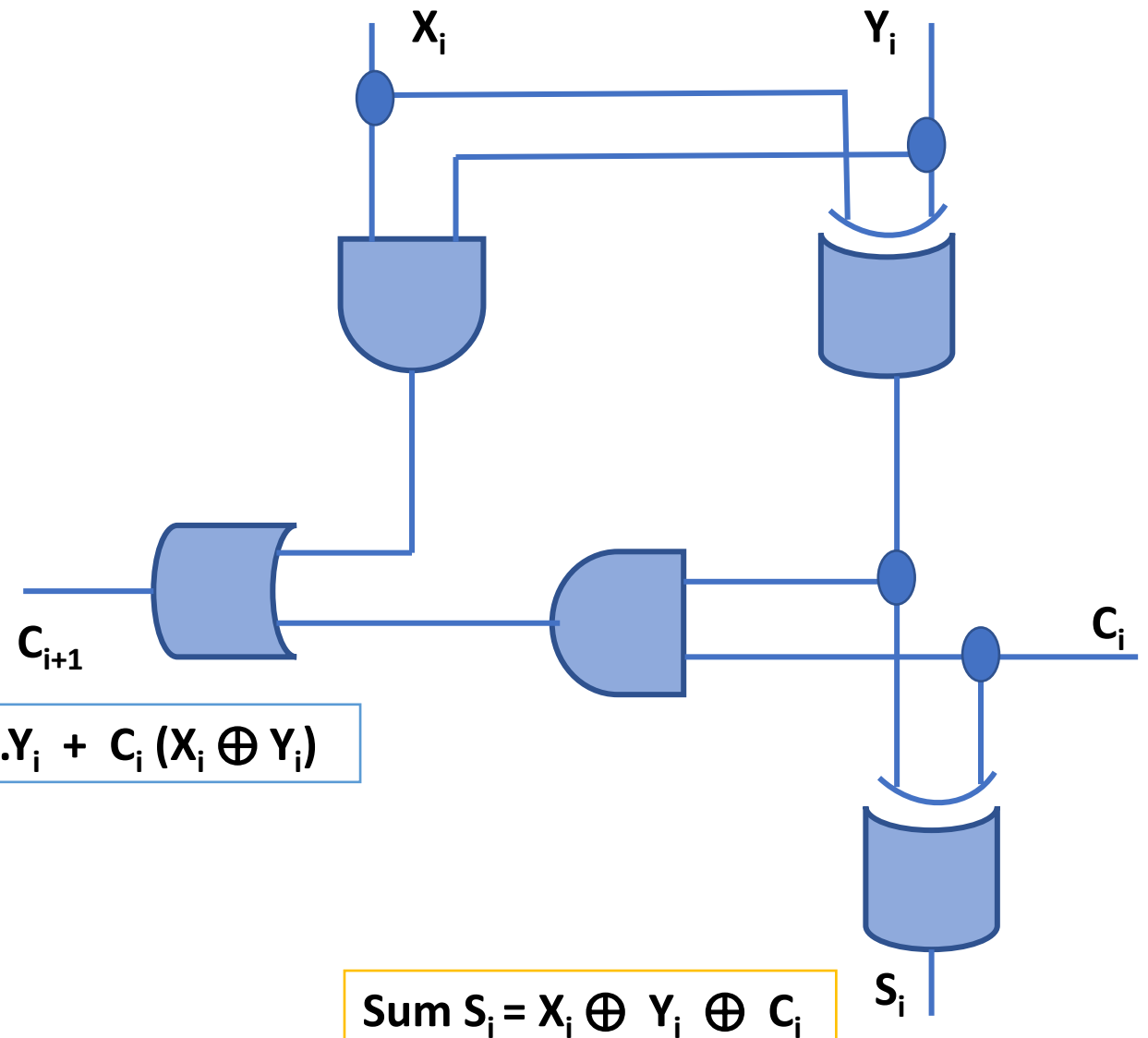
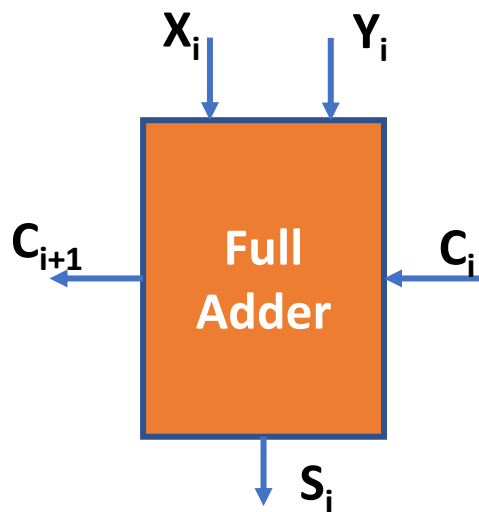
$$C_{i+1} = X_i \cdot Y_i + C_i (X_i \oplus Y_i)$$



Full Adder Circuit

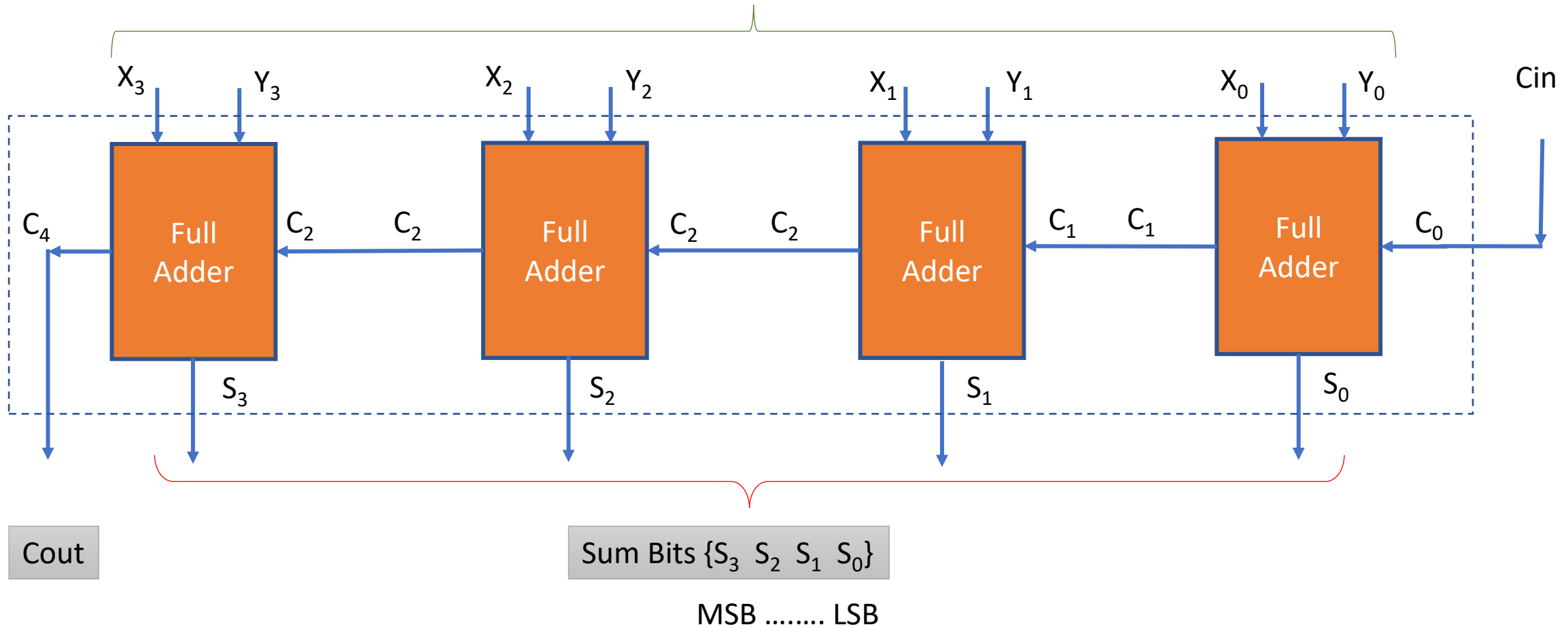
$$\text{Sum } S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i \cdot Y_i + C_i (X_i \oplus Y_i)$$



Multiple Bits – Ripple Carry Adder

Input Numbers $\{X_3 X_2 X_1 X_0\}$ and $\{Y_3 Y_2 Y_1 Y_0\}$, Carry Input at Cin



Timing Issues in Ripple Carry Adders

❖ Delay between Y_i and C_{i+1}
 = 1 EXOR + 1 AND + 1 OR (= Critical Path)

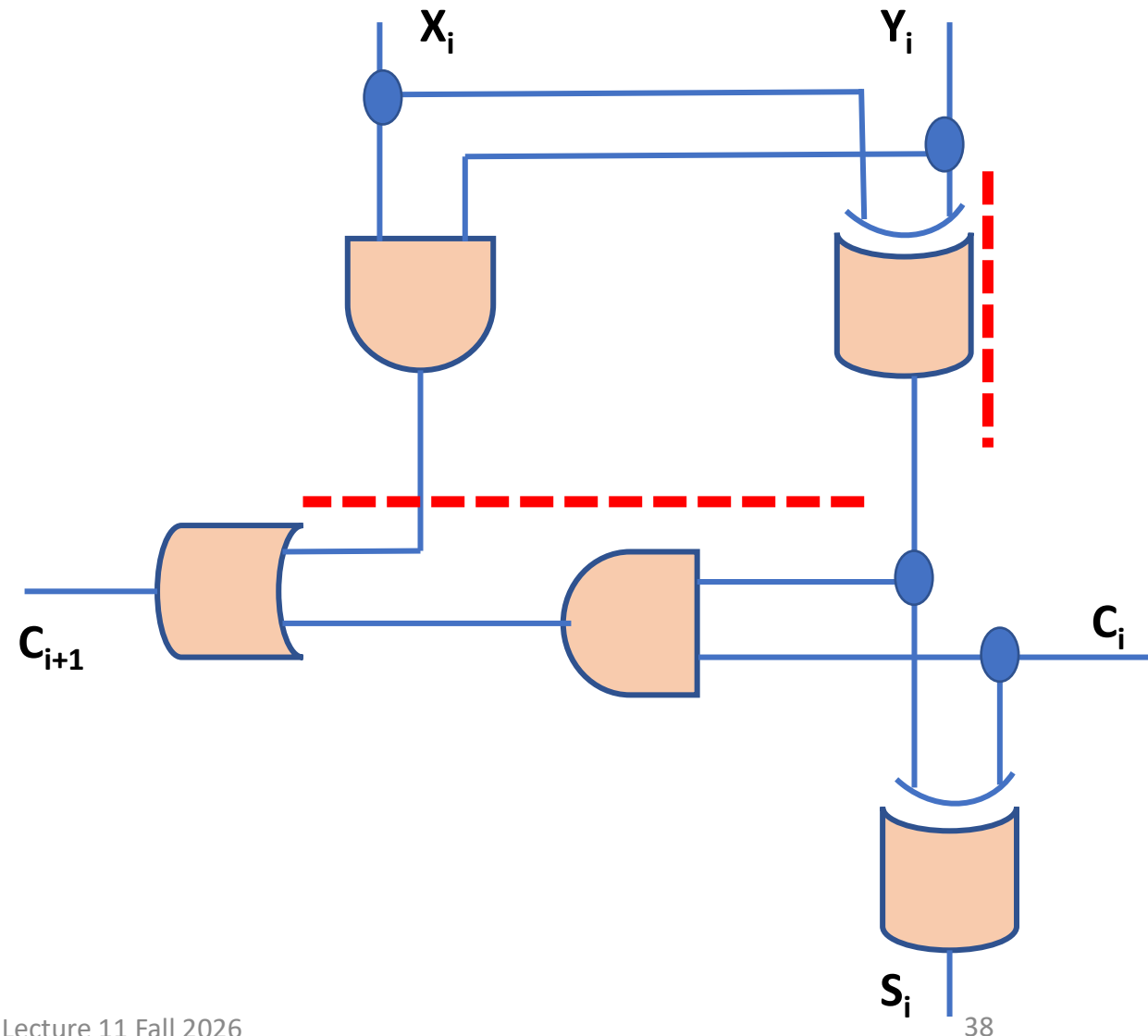
❖ Suppose delays through gates are:
 EXOR = 20ns, AND = 10ns, OR = 10ns

❖ Then Delay between Y_i and C_{i+1}
 = 20ns + 10ns + 10ns = 40ns

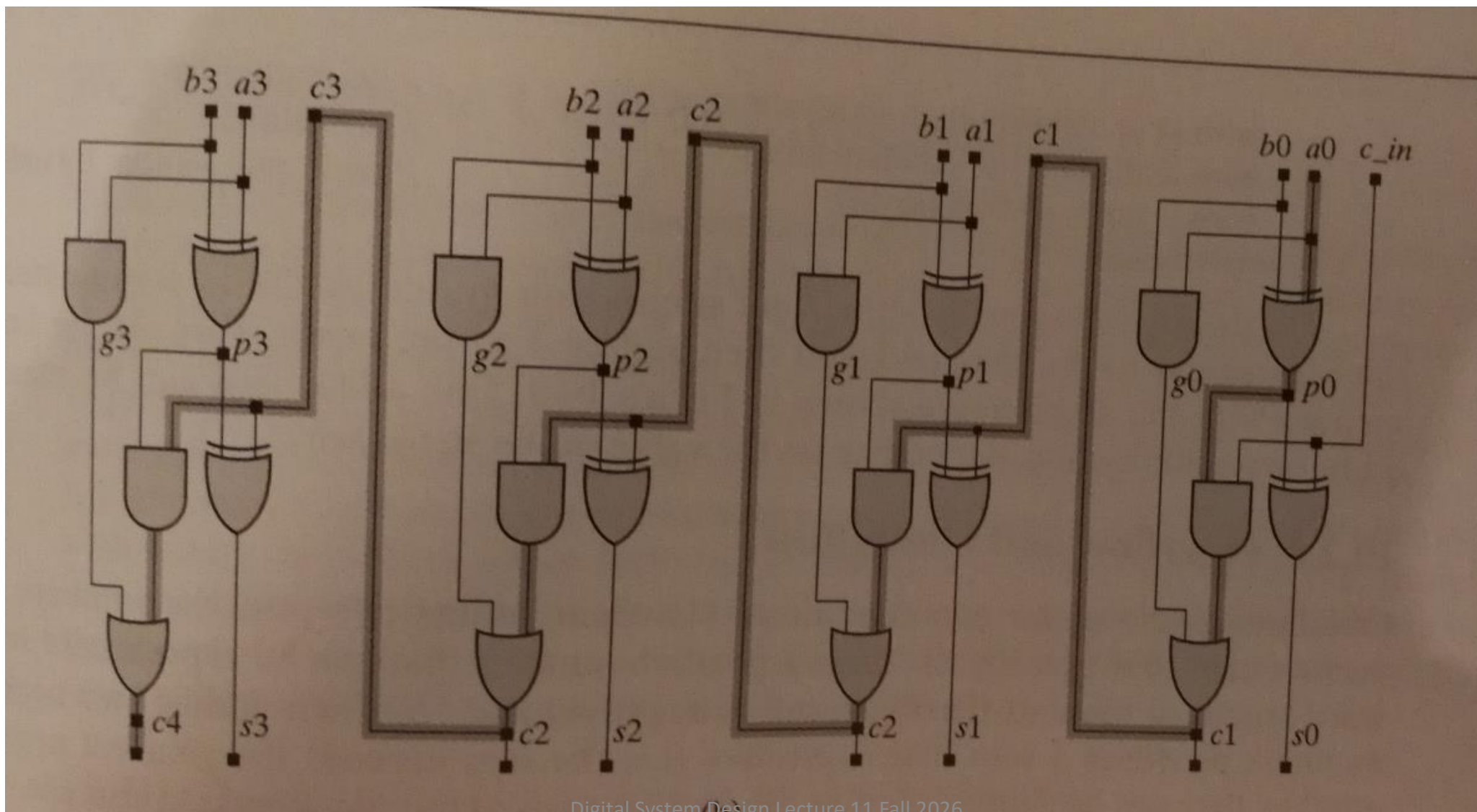
❖ Worst case delay through N adders
 = $N \times 40\text{ns}$

❖ For 32-bit data;
 Delay through 32-bit RCA =
 $32 \times 40\text{ns} = 1280\text{ns} = 1.28\mu\text{s}$

❖ Corresponds to Maximum Data Throughput
 = $1/(1.28\mu\text{s}) = 0.78\text{ MHz}$ (too slow in today's terms)



Delay of Ripple Carry Full Adder



Compare with Critical Path of Full Adder

REVIEW

